极客时间

部署上线与运维 — 完结篇

从“能用”到“上线”— 四周总结 + V4 里程碑 + 毕业设计

第 4 周 · 第 16 节（FINAL）

讲师：黄佳

极客时间黄佳老师出品

极客时间

极客时间

产品形态有了
但它 7×24 运行在哪里?

上线 = Docker + 云服务器 + Cron 编排 + 监控告警 + 回滚方案

你真的需要 Docker 吗?

你的目标	16-1 Docker	推荐路径
自己用够了,关机一会儿无所谓	不用	已达成,直接毕业
采集要 7×24 不掉线(我睡觉也得采)	不用	GitHub Actions(daily-collect-v4 已配)
Bot 也要 7×24 在线(半夜有人问也答)	必做	云服务器 + Docker
要 share 给别人用 / 想做产品	必做	Docker 镜像 + 上线 Checklist

极客时间

不是“必须把 Bot 装进容器”是 3 个工程化思维

部署边界 · 环境一致性 · Checklist 文化

云服务器选择 — 一些方案

极客时间

方案	价格	配置	适合场景	推荐指数
Hetzner VPS	~$5/月（¥35）	2C/2G/20G	海外访问 + 低成本	★★★★★
阿里云 ECS	~¥50/月	2C/2G/40G	国内访问为主	★★★★
Railway	按用量计费	弹性资源	低频使用 + 零运维	★★★

极客时间

极客时间

Docker 部署

一次构建，到处运行 — 告别「我本地能跑」

极客时间

极客时间

Dockerfile — 多阶段镜像构建

Dockerfile | dockerfile

```
# 构建层：装 Python 依赖
FROM python:3.12-slim AS builder
WORKDIR /build
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc && \
    rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r
requirements.txt

# 运行层：只要必要文件
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
    CMD python -c "import sys; sys.exit(0)"

CMD ["python", "-m", "bot.knowledge_bot"]
```

Dockerfile | dockerfile

```
# 拷贝代码（V3 工作流 + V4 分发 + OpenClaw 配置）
COPY workflows/ ./workflows/
COPY patterns/ ./patterns/
COPY pipeline/ ./pipeline/
COPY distribution/ ./distribution/
COPY bot/ ./bot/
COPY openclaw/ ./openclaw/
COPY scripts/ ./scripts/
COPY AGENTS.md .

# 非 root 用户 + 健康检查
RUN groupadd -r appuser && useradd -r -g appuser
appuser && \
    chown -R appuser:appuser /app
USER appuser
HEALTHCHECK --interval=60s --timeout=10s --
retries=3 \
    CMD python -c "import sys; sys.exit(0)"

CMD ["python", "-m", "bot.knowledge_bot"]
```

极客时间黄佳老师出品

docker-compose.yml — 服务编排

极客时间

docker-compose.yml | yaml

```yaml
version: "3.8"

services:
  knowledge-base:
    build: .
    container_name: ai-knowledge-base
    restart: unless-stopped
    env_file: .env
    volumes:
      - ./knowledge:/app/knowledge
      - ./logs:/app/logs
    environment:
      - TZ=Asia/Shanghai
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: "1.0"
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

极客时间黄佳老师出品

极客时间

Cron 定时任务编排

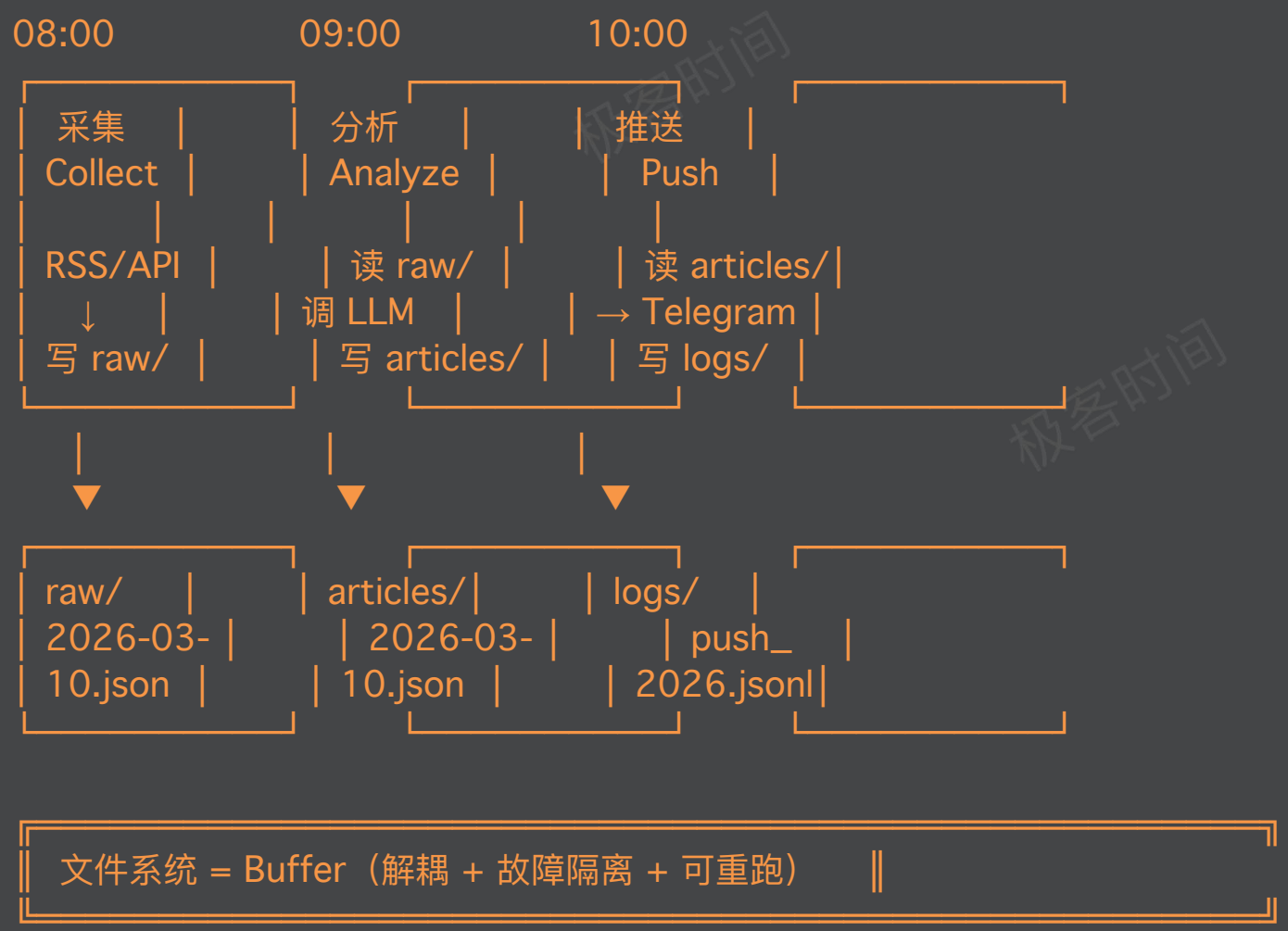
缓冲设计 — 每个任务只做一件事

极客时间

极客时间

极客时间黄佳老师出品

Cron 缓冲编排 — 三步分时执行



极客时间黄佳老师出品

Crontab 配置

docker-compose.yml 的 pipeline 服务（cron -f） | bash

```
# docker-compose.yml 里的 pipeline 服务
# 用 cron -f 把下面的 crontab 加载进去

SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin

# 每日早上 08:00 跑完整流水线
# pipeline.pipeline 内部串起 V3 LangGraph + V4 分发
0 8 * * * cd /app && python -m pipeline.pipeline \
    >> /app/data/pipeline.log 2>&1

# 每日晚上 20:00 再跑一次（覆盖全天数据）
0 20 * * * cd /app && python -m pipeline.pipeline \
    >> /app/data/pipeline.log 2>&1

# 只发简报（用已有的 articles，不重跑采集）
# 每天中午 12:00 补发
0 12 * * * cd /app && python daily_digest.py \
    >> /app/data/digest.log 2>&1

# 日志清理 每月 1 号 03:00
0 3 1 * * find /app/data -name '*.log' -mtime +30 -delete
```

极客时间

监控与告警

看不见的 Bug 才是最可怕的 Bug

极客时间

极客时间

极客时间黄佳老师出品

四级告警策略

级别	触发条件	动作	示例
INFO	管线正常完成	写日志	今日采集 15 条数据
WARN	预算 > 70% / 耗时 > 5min	日志 + 控制台	本次管线成本 ¥0.07
ERROR	预算超限 / 输出不安全	日志 + 停止执行	CostGuard 预算超限
CRITICAL	熔断器触发 / 安全攻击	日志 + 停止 + Telegram	检测到 Prompt Injection

极客时间黄佳老师出品

健康检查 — Docker HEALTHCHECK

极客时间

- **Dockerfile 里的极简探活**
- HEALTHCHECK 用单行 Python，不用独立脚本
- CMD python -c "import sys; sys.exit(0)"

- **进阶版（可选）：检查关键依赖**
- 检查 LLM_API_KEY 环境变量是否已配置
- 检查 knowledge/articles 目录是否可写
- 检查磁盘剩余空间 > 1GB

- **调用链**：Docker 每 60s → HEALTHCHECK → CMD
- 失败 3 次标记 unhealthy → docker-compose 重启容器

极客时间

上线前 Checklist — 8 项必查

- ☐ 1. PI Keys 环境变量 — .env 注入，.gitignore 排除，权限 600
- ☐ 2. 权限策略 — 每个 Agent 最小必要权限
- ☐ 3. 备份策略 — knowledge/ 每日备份 + Git push + 镜像版本
- ☐ 4. 日志轮转 — 单文件 < 10MB，30 天自动清理
- ☐ 5. 成本预算 — CostGuard daily/pipeline budget 已设置
- ☐ 6. 版本固定 — requirements.txt 有版本号，不用 :latest
- ☐ 7. 测试通道 — 单元测试通过 + Eval 通过 + 手动验证
- ☐ 8. 回滚方案 — 镜像版本号回滚 + 数据备份恢复步骤

极客时间黄佳老师出品

极客时间

四周进化回顾

V1 → V2 → V3 → V4 · 从工具使用者到系统架构师

极客时间黄佳老师出品

V1 → V2 → V3 → V4 四版演进

极客时间

Week 1
V1 编码自动化

AGENTS.md

Sub-Agents

Skills

Week 2
V2 采集自动化

Pipeline

Hooks 校验

CI/CD

Week 3
V3 质量自动化

LangGraph

Review Loop

CostGuard

Week 4
V4 服务自动化

Bot 对话查询

多渠道分发

标签订阅

Docker 部署

关键跃迁：

V2 采集自动化（数据进来了，死的）

→ V4 服务自动化（数据活了，用户能查/订/收推送）

四周能力蜕变

极客时间

Week 1 的你

- 聊天式 prompt
- 全手动操作
- 质量靠感觉
- 不知道花了多少钱
- API Key 裸写代码里
- 只会本地跑
- 想到哪写到哪

Week 4 的你

- 结构化 Agent 角色 + Skill
- Pipeline + CI/CD + Cron
- Reviewer 审核 + 三层测试
- CostGuard + 模型路由
- 三道防线 + 最小权限
- Docker + 云服务器 + 监控
- 架构先行 + 渐进迭代

十大工程思想

贯穿四周的底层方法论

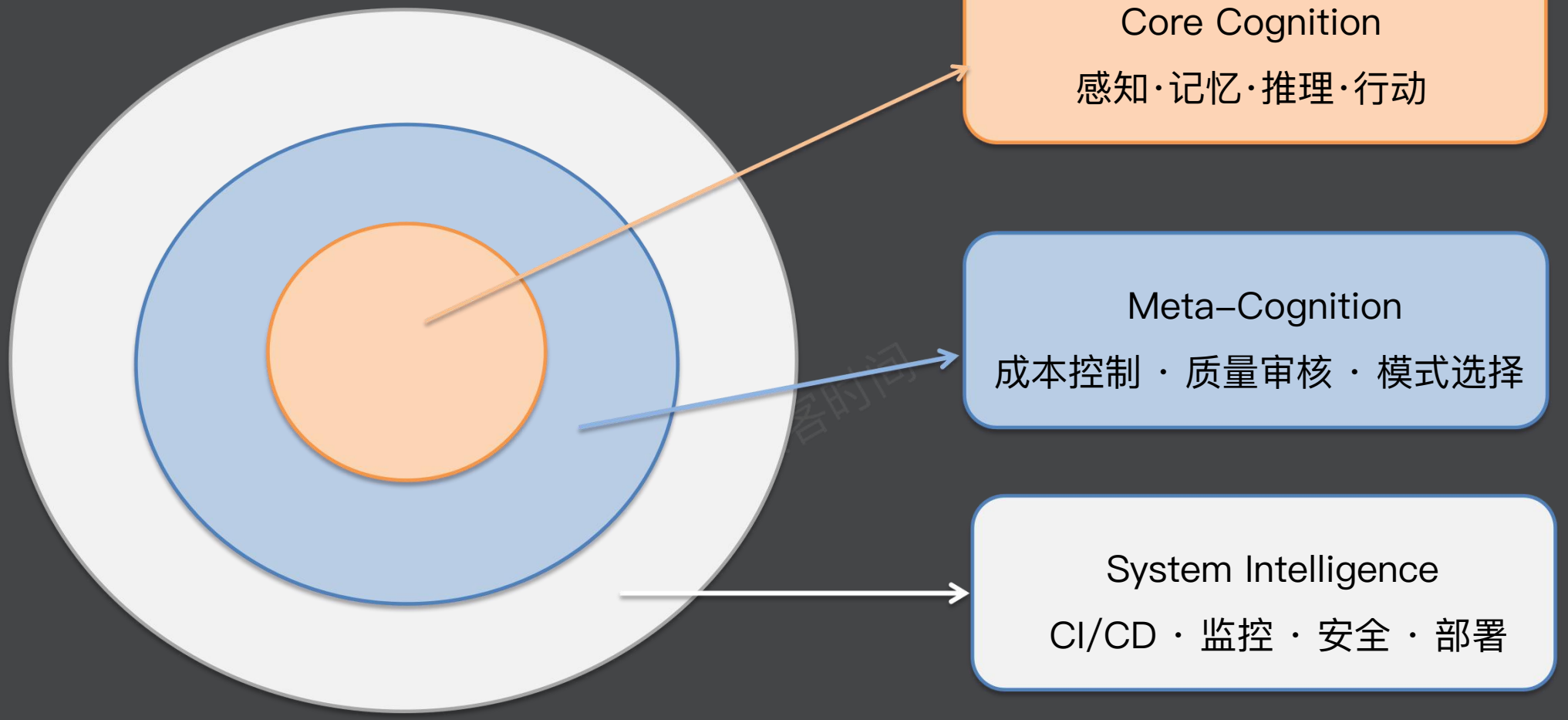
极客时间

极客时间

极客时间

极客时间黄佳老师出品

Agent 能力栈



Core Cognition → Meta–Cognition → System Intelligence
三环同心圆，由内而外逐层扩展

极客时间黄佳老师出品

十大工程思想（1–5）

极客时间

序号	思想	核心理念	课程中的应用
1	设计反转	人写规范，AI 写代码	SDD 三层规范
2	无状态推理 +有状态编排	LLM 无状态 Harness 有状态	规范→Harness→模型
3	声明式配置 优先	描述要什么 不描述怎么做	AGENTS.md SKILL.md
4	反馈驱动 循环	AI 输出不确定 必须有反馈	Hooks + Reviewer + Eval
5	Token 经济学	Token Debt/ Leverage/Dividend	CostGuard + 三档路由

十大工程思想（6–10）

极客时间

序号	思想	核心理念	课程中的应用
6	上下文窗口 隔离	每个 Agent 只看 需要的信息	Sub–Agents 独立上下文
7	Pattern Selection Card	ASSESS→ROUTE →SELECT	Router vs Supervisor
8	报告式 通信	Agent 间用 JSON 通信	Pipeline 每步 输出 JSON
9	按需上下文 加载	Skill description = 路由触发器	用到什么 加载什么
10	风险分层	不同风险等级 不同策略	四级告警 + Checklist

极客时间

佳哥寄语

极客时间

动手实践
敢想，还要敢做
不要拘泥于昨天的自己，同时也要务实

从实际出发

极客时间

毕业设计

GitHub 仓库 + README + 截图 — 你的 V4 作品
提交时间：截止到 2026.5.10 24:00

极客时间黄佳老师出品

毕业设计 — 提交要求

极客时间

1. GitHub 仓库

- V1–V4 全部代码 + 完整目录结构

2. README.md

- 项目名称 + 一句话描述 + 架构图 + 快速开始

3. 配置文件

- .env.example + .gitignore + requirements.txt

4. 运行截图（≥3 张）

- 管线运行成功 + Telegram 推送 + 日志/成本统计

5. Git 历史（≥10 commits）

- 有意义的 commit 信息，反映渐进开发过程

极客时间

评分标准

维度	权重	满分标准
代码完整性	30%	V1–V4 全部模块完成
工程质量	30%	有测试、日志、成本控制、安全防护
文档质量	20%	README 清晰 + 架构图准确 + 快速开始可用
创新加分	20%	自定义 Skill、多数据源、特色功能

极客时间

从工具使用者到系统架构师

AI 时代不缺会用工具的人
缺的是能把 AI 工具
变成 AI 系统的工程师

四周的旅程到此结束。去构建你自己的 AI 系统吧！

极客时间黄佳老师出品

极客时间

本节小结

极客时间

极客时间

下一节：课程完结，学习不止一 去构建你自己的 AI 系统！